

# Problem Set 4

CMSC 27200: Theory of Algorithms

Assigned February 3, Due February 14

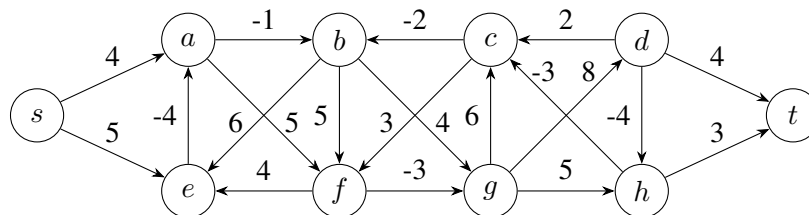
## Problem 0: Collaborators and Outside Sources

Who did you collaborate with?

Did you use any outside sources? If so, which sources did you use?

## Problem 1: Bellman-Ford

- (a) Run the Bellman-Ford algorithm on the following graph. For your answer, for each  $k$  and each vertex  $v \in V(G)$ , you should provide the length of the shortest walk from  $s$  to  $v$  which uses at most  $k$  edges. You may stop when  $k = 10$  or the answers for  $k$  are the same as the answers for  $k - 1$ .



What are the shortest path(s) from  $s$  to  $t$ ?

- (b) Now let's say the length of the edge  $(a, f)$  is decreased from 5 to 1. Run the Bellman-Ford algorithm for the graph with this modified edge length. What is happening here and why?

## Problem 2: Undominated Chessboxers

Acknowledgement: This problem is due to Andy Drucker:

Let's say that there are  $n$  chessboxers where chessboxer  $i$  has a skill level  $x_i$  in chess and a skill level  $y_i$  in boxing. We say that chessboxer  $j$  dominates chessboxer  $i$  if  $x_i < x_j$  and  $y_i < y_j$  (i.e., chessboxer  $j$  is better than chessboxer  $i$  in both chess and boxing).

- (a) Give an  $O(n \log n)$  time algorithm to find the set of chessboxers who are not dominated by any other chessboxer. In other words, find the set  $S = \{i : \text{For all } j \in [n] \setminus \{i\}, x_i \geq x_j \text{ or } y_i \geq y_j\}$ . Explain why your algorithm is correct and takes  $O(n \log n)$  time.

Hint is available.

Note: as described in the divide and conquer rubric, it is possible to earn an exemplary grade for your algorithm.

- (b) Extra credit: Now consider the extension where a third activity is added and a player with skill levels  $(x, y, z)$  is only dominated by a player with skill levels  $(x', y', z')$  if  $z' > z$ ,  $y' > y$ , and  $x' > x$ . Give an  $O(n \log n)$  time algorithm to find the set of players who are not dominated by any other player and explain why your algorithm is correct.

Hint is available. Note: a correct answer increases your score by 1, there is no penalty for an incorrect or incomplete answer. As a warning, this problem is tough.

### Problem 3: Max-Weight Zigzag Walk

You are given an  $N$ -by- $N$  grid of cells  $(x, y) \in [N] \times [N]$ , with a (possibly negative) weight  $w_{(x,y)} \in \mathbb{R}$  at each cell  $(x, y) \in [N] \times [N]$ . You start at the Southwest corner  $(1, 1)$  of this grid, and would like to take a *zigzag* walk to reach the Northeast corner  $(N, N)$ . This means that in each step of this walk, you are only allowed to move to the adjacent cell either to the east or to the north, i.e., suppose you are currently at cell  $(x, y)$ , you can either move to  $(x + 1, y)$  or  $(x, y + 1)$ , and you are not allowed to leave the grid. Give an algorithm to find a zigzag walk of maximum weight, where the weight of the walk is the sum of the weights at all cells visited by the walk.

- (a) What subproblems should we solve in order to determine the maximum weight of a zigzag walk?
- (b) What is the recurrence relation for solving these subproblems? What are the base case(s)?
- (c) What order should solve the subproblems in? How can we use the solutions to these subproblems to find a zigzag walk of maximum weight?

Note: There are multiple valid orders and it is sufficient to give one of them.

- (d) Give a rigorous proof that the recurrence relation is correct.
- (e) What is the runtime of your algorithm?
- (f) Reflection, not for credit: Is this problem similar to a dynamic programming problem you saw in class?

Note: As described in the rubric, parts a, b, and c will be graded together as the dynamic programming algorithm and you will earn an exemplary grade for this if everything is correct.

## Problem 4: Job Management (Rephrasing of Problem 6.20 in Kleinberg-Tardos)

You have  $n$  jobs to work on and a total of  $H$  hours to spend on these jobs. The reward that you get for each job depends on how many hours you work on the job (you like dividing your time into hours, so you will spend a whole number of hours on each job). This dependence can be complicated but is monotonic, i.e. if you spend more time on a job then this can only increase your reward. Given the reward functions, how can you allocate your time to obtain the maximum total reward?

For example, if  $n = 3$ ,  $H = 5$ , and the rewards are as follows

1. The rewards for spending 0, 1, 2, 3, 4, 5 hours on job 1 are 0, 1, 3, 6, 9, 10.
2. The rewards for spending 0, 1, 2, 3, 4, 5 hours on job 2 are 0, 4, 5, 5, 6, 12.
3. The rewards for spending 0, 1, 2, 3, 4, 5 hours on job 3 are 0, 3, 7, 8, 9, 10.

then it is best to spend 2 hours on job 1, 1 hour on job 2, and 2 hours on job 3 for a total reward of 14.

We can state this problem as follows. We have  $n$  non-decreasing functions  $f_1, \dots, f_n$  from  $\{0, 1, \dots, H\} \rightarrow \mathbb{R}^+$  and we want to find  $t_1, \dots, t_n$  such that  $\sum_{j=1}^n t_j = H$  and  $\sum_{j=1}^n f_j(t_j)$  is maximized.

- (a) What subproblems should we solve in order to determine the maximum total reward you can obtain?
- (b) What is the recurrence relation for solving these subproblems? What are the base case(s)?
- (c) What is the order that we should solve the subproblem in? How can we use the solutions to these subproblems to find an allocation of your time to give the maximum possible total reward?

Note: There are multiple valid orders and it is sufficient to give one of them.

- (d) Give a rigorous proof that the recurrence relation is correct.
- (e) What is the runtime of your algorithm?  
Note: This runtime should be  $\text{poly}(n, H)$ .
- (f) Reflection, not for credit: Is this problem similar to a dynamic programming problem you saw in class?

Note: As described in the rubric, parts a, b, and c will be graded together as the dynamic programming algorithm and you will earn an exemplary grade for this if everything is correct.

Hints:

2a. If  $i_1, \dots, i_m$  are the undominated players and  $x_{i_1} < \dots < x_{i_m}$ , what can we say about  $y_{i_1}, \dots, y_{i_m}$ ?

2b. It may be helpful to divide the players into 8 subgroups based on whether their skill level in each activity is above average or below average. Is there additional information about each subgroup (beyond the set of players who are undominated within the subgroup) which would be helpful for comparing people in different subgroups? Given this information for the subgroups, can you obtain this information for the entire group?